

The mathematics behind the neural network chain ladder and its chain-ladder start

Explainer for `R/nn_chain_ladder.R` and the scripts `NN chain ladder SynthETIC fit.R`
and `NN chain ladder SynthETIC analysis.R`

Prepared for Francis

2 October 2026

Contents

Notation used in this document	3
1 What this document covers	4
2 Set-up: cells and cumulative payments	4
3 The chain-ladder model with features	4
3.1 Model assumptions	4
3.2 The loss function	5
3.3 The homogeneous model is Mack's chain-ladder factor	5
4 The network	6
4.1 One hidden layer (the paper)	6
4.2 Several hidden layers	6
4.3 Feature coding	7
5 The loss as Keras sees it	7
5.1 Response and volume	7
5.2 The unit of the payments	7
6 Gradients	7
6.1 Derivative in the output	7
6.2 Output layer	8
6.3 Hidden layers (back-propagation)	8
7 The chain-ladder start	8
7.1 Definition	8
7.2 Property 1: the start is the chain ladder	9
7.3 Property 2: the gradient at the start	9
7.4 Property 3: curvature in the level	10
7.5 What the first steps do	10
7.6 Comparison with the paper's random start	10
8 Early stopping and the stopping rule of the chain-ladder start	11
8.1 Training and validation rows	11
8.2 Early stopping	11

8.3	Keeping the start	11
9	Optimisers	12
9.1	Update rules	12
9.2	Step size of the adaptive methods	12
9.3	Why the unit of the payments matters for SGD only	12
10	The balance correction (S4)	13
11	The nagging predictor	13
11.1	Definition	13
11.2	Why averaging helps	14
11.3	The reserve of a nagging predictor	14
12	Reserves and the zero claims features	15
12.1	Part 1: cells with a positive diagonal	15
12.2	Part 2: zero claims features	15
12.3	The three rules of the code	16
13	Reading the fitted factors: marginal averages	16
14	Where each formula is in the code	16
15	Numerical check of the formulas	16

Notation used in this document

Symbol	Meaning	Section
I, J	number of accident years, last development year; here $I = 20$, $J = I - 1 = 19$	2
i, j	accident year $1, \dots, I$; development year $0, \dots, J$	2
$\mathbf{x}$	feature value of a claim (its five covariates); as network input a vector in $\mathbb{R}^d$, $d = 14$	2, 4
$C_{i,j}(\mathbf{x})$	cumulative payments up to development year j of all claims of accident year i with feature $\mathbf{x}$	2
$C_{i,j}$	the same summed over all feature values (the ordinary triangle)	2
$f_{j-1}(\mathbf{x})$	chain-ladder (CL) factor from development year $j - 1$ to j for feature $\mathbf{x}$	3
$\hat{f}_{j-1}$	homogeneous CL factor: the same for every $\mathbf{x}$	3
$\mathcal{L}_j, n_j$	learning cells of development period j ; their number	3
$\mathcal{T}_j, \mathcal{V}_j$	training rows (first 90% of $\mathcal{L}_j$) and validation rows (last 10%)	8
$L_j(f)$	weighted squared loss of development period j	3
$q_1, \dots, q_M$	neurons of the M hidden layers	4
$W^{(m)}, \mathbf{b}^{(m)}$	weights and intercepts of hidden layer m	4
$\mathbf{z}^{(m)}(\mathbf{x})$	activations of hidden layer m (tanh neurons)	4
β_0, β	intercept and weights of the output neuron	4
$a(\mathbf{x})$	output before the exponential: $a = \log f$	4
θ	all parameters of one network	4
y, w	response and volume handed to Keras: $y = C_{i,j}/\sqrt{C_{i,j-1}}$, $w = \sqrt{C_{i,j-1}}$	5
u	unit of the payments in the fit, $u = 10^6$	5
δ_r	derivative of the loss in the output a of row r	6
η	learning rate of the optimiser	9
g_t	gradient of the training loss at step t	9
c_j	balance correction factor of development period j	10
$\bar{f}_{j-1}(\mathbf{x})$	nagging predictor: average of the CL factors of K networks	11
$g_j^{(i)}$	CL factor of the zero claims features of accident year i	12
$\hat{R}$	reserve estimate	12

Abbreviations: CL, chain ladder; NN, neural network; AY, accident year; SGD, stochastic gradient descent. “The paper” is Wuthrich (2018), *Neural networks applied to chain-ladder reserving*, European Actuarial Journal 8:407-436. Its equation numbers are quoted as your code comments quote them ((3.1), (3.2)-(3.3), (3.9), (5.1), (6.1), (6.2), Listing 2, Section 4.2, Proposition 2.2); I have not re-checked them against the paper for this explainer.

1 What this document covers

This explainer writes out, formula by formula, what the code of the neural network chain ladder computes on the SynthETIC portfolio: the model, the loss, the gradients, the optimisers, the stopping rule, the reserves and the nagging predictor. Most space goes to the **chain-ladder start** (the training called `cl_start`, sensitivity run S3), because that is the variant that works on this data and because its behaviour can be derived exactly at the starting point.

Three things in the code are not in the paper and are marked as such where they appear: the chain-ladder start with its stopping rule (Section 7 and 8), the balance correction (Section 10), and the networks with several hidden layers together with the grid of optimisers (Sections 4 and 9). The nagging predictor (Section 11) is from Richman and Wuthrich (2020), *Nagging predictors*, Risks 8(3):83.

How the maths was checked. The formulas of Sections 3, 5, 6 and 7 were recomputed in base R, without Keras, on the learning cells of development period $j = 2$ of your data and compared with what the scripts saved. The results are in Section 15. In short: the gradient formulas agree with finite differences to a relative error of 10^{-7} , the statements about the chain-ladder start hold to machine precision, and the parameter counts agree with Keras's.

2 Set-up: cells and cumulative payments

The portfolio has $I = 20$ accident years $i = 1, \dots, I$ and development years $j = 0, \dots, J$ with $J = I - 1 = 19$ (development year j here is development year $j + 1$ of the Mack and bCCNN scripts). Every claim carries a feature value $\mathbf{x}$: its combination of the five covariates Legal Representation (2 labels), Injury Severity (6), Age of Claimant (5), Vehicle type (4) and Business use (2). Of the 480 combinations, 226 occur.

A **cell** is a pair $(i, \mathbf{x})$. Its cumulative payments are

$$C_{i,j}(\mathbf{x}) = \sum_{\text{claims of AY } i \text{ with feature } \mathbf{x}} \sum_{l=0}^j \text{payments in development year } l, \quad C_{i,j} = \sum_{\mathbf{x}} C_{i,j}(\mathbf{x}).$$

There are 3,342 cells. At the end of year I the cell $(i, \mathbf{x})$ is observed up to development year $I - i$; the observed set is $\{(i, j, \mathbf{x}) : i + j \leq I\}$ and $C_{i,I-i}(\mathbf{x})$ is the **diagonal** value (`c_diag`). The quantity to predict is

$$R = \sum_{i=2}^I \sum_{\mathbf{x}} (C_{i,J}(\mathbf{x}) - C_{i,I-i}(\mathbf{x})),$$

the payments still to come up to development year J . In the simulated data it is known: $R^{\text{true}} = 9,626.8$ million.

In the code the matrix `cum` holds $C_{i,j}(\mathbf{x})$ with one row per cell and columns `cum_0`, $\dots$, `cum_19`.

3 The chain-ladder model with features

3.1 Model assumptions

The paper extends Mack's distribution-free chain ladder by letting the factors depend on the feature (the paper's Section 2). For every cell, with the cells independent,

$$\mathbb{E}[C_{i,j}(\mathbf{x}) | C_{i,j-1}(\mathbf{x})] = f_{j-1}(\mathbf{x}) C_{i,j-1}(\mathbf{x}), \quad \text{Var}(C_{i,j}(\mathbf{x}) | C_{i,j-1}(\mathbf{x})) = \sigma_{j-1}^2 C_{i,j-1}(\mathbf{x}), \quad j = 1, \dots, J. \quad (1)$$

Iterating the first equation gives the predictor of the ultimate of a cell with a positive diagonal value (the paper’s Proposition 2.2),

$$\hat{C}_{i,J}(\mathbf{x}) = C_{i,I-i}(\mathbf{x}) \prod_{j=I-i}^{J-1} f_j(\mathbf{x}). \quad (2)$$

Mack’s ordinary chain ladder is the special case $f_{j-1}(\mathbf{x}) \equiv f_{j-1}$.

3.2 The loss function

One function $f_{j-1}(\cdot)$ is estimated per development period $j = 1, \dots, J$, each on its own **learning cells**

$$\mathcal{L}_j = \{(i, \mathbf{x}) : i \leq I - j, C_{i,j-1}(\mathbf{x}) > 0\}, \quad n_j = |\mathcal{L}_j|.$$

The condition $i \leq I - j$ says that $C_{i,j}(\mathbf{x})$ is observed, and $C_{i,j-1}(\mathbf{x}) > 0$ that there is a volume to develop. On SynthETIC $n_1 = 2,895$, $n_2 = 2,952$, and the number falls to $n_{19} = 159$.

The factor is estimated by minimising the weighted squared loss (the paper’s (3.1), reported as L'_j in (6.1))

$$L_j(f) = \sum_{(i,\mathbf{x}) \in \mathcal{L}_j} \frac{(C_{i,j}(\mathbf{x}) - f(\mathbf{x}) C_{i,j-1}(\mathbf{x}))^2}{C_{i,j-1}(\mathbf{x})}. \quad (3)$$

The weight $1/C_{i,j-1}(\mathbf{x})$ is the inverse of the variance in (1) up to the constant σ_{j-1}^2 : every squared residual is divided by its own variance, so (3) is the weighted least squares criterion that belongs to model (1). In the code it is the element `loss` of `nncl_fit()`.

3.3 The homogeneous model is Mack’s chain-ladder factor

If $f(\mathbf{x}) \equiv f$ is a single number, (3) is a quadratic in f . Write $c_r = C_{i,j-1}(\mathbf{x})$ and $d_r = C_{i,j}(\mathbf{x})$ for the rows r of $\mathcal{L}_j$. Then

$$\frac{\partial L_j}{\partial f} = -2 \sum_r \frac{(d_r - f c_r) c_r}{c_r} = -2 \sum_r (d_r - f c_r) = 0 \quad \Longleftrightarrow \quad \hat{f}_{j-1} = \frac{\sum_r d_r}{\sum_r c_r} = \frac{\sum_{\mathcal{L}_j} C_{i,j}(\mathbf{x})}{\sum_{\mathcal{L}_j} C_{i,j-1}(\mathbf{x})}, \quad (4)$$

and $\partial^2 L_j / \partial f^2 = 2 \sum_r c_r > 0$, so this is the minimum. The estimator (4) is the volume-weighted CL factor: this is the “homogeneous model” of the first block of the paper’s Table 5 and the columns `f_hom` and `loss` of `nncl_synthetic_homogeneous.rds`.

It is almost, but not exactly, Mack’s factor on the aggregate triangle, $\sum_{i \leq I-j} C_{i,j} / \sum_{i \leq I-j} C_{i,j-1}$. The difference is the cells with $C_{i,j-1}(\mathbf{x}) = 0$ and $C_{i,j}(\mathbf{x}) > 0$: they are in Mack’s numerator but not in $\mathcal{L}_j$. Their payments are what the zero claims features of Section 12 are for. On your data:

j	1	2	3	4	10
Mack's factor on the triangle	2.53686	1.32671	1.13806	1.07402	1.00585
$\hat{f}_{j-1}$ on all learning cells, (4)	2.52775	1.32628	1.13804	1.07402	1.00585
$\hat{f}_{j-1}$ on the training rows $\mathcal{T}_j$ (Section 7)	2.53498	1.32669	1.13756	1.07428	1.00503

4 The network

4.1 One hidden layer (the paper)

The feature $\mathbf{x}$ enters as $d = 14$ dummy variables (Section 4.3). With q hidden neurons the paper's network (3.2)-(3.3), Listing 2, is

$$z_k(\mathbf{x}) = \tanh\left(b_k + \sum_{l=1}^d W_{k,l} x_l\right), \quad k = 1, \dots, q, \quad f(\mathbf{x}) = \exp\left(\beta_0 + \sum_{k=1}^q \beta_k z_k(\mathbf{x})\right). \quad (5)$$

The exponential output makes the factor positive and makes the model a log-linear regression on the learned features z_k : $\log f(\mathbf{x}) = \beta_0 + \boldsymbol{\beta}^\top \mathbf{z}(\mathbf{x})$. The number of parameters is $q(d+1) + (q+1)$; for $q = 20$ that is $20 \cdot 15 + 21 = 321$.

4.2 Several hidden layers

`nnc1_model()` now takes a vector $\mathbf{q} = \mathbf{c}(\mathbf{q}_1, \dots, \mathbf{q}_M)$. With $\mathbf{z}^{(0)} = \mathbf{x}$ and $q_0 = d$,

$$\mathbf{z}^{(m)} = \tanh\left(W^{(m)} \mathbf{z}^{(m-1)} + \mathbf{b}^{(m)}\right), \quad m = 1, \dots, M, \quad a(\mathbf{x}) = \beta_0 + \boldsymbol{\beta}^\top \mathbf{z}^{(M)}(\mathbf{x}), \quad f(\mathbf{x}) = e^{a(\mathbf{x})}, \quad (6)$$

where $W^{(m)}$ is a $q_m \times q_{m-1}$ matrix and $\tanh$ acts on each component. The parameter count is

$$|\theta| = \sum_{m=1}^M q_m (q_{m-1} + 1) + (q_M + 1).$$

hidden layers	calculation	parameters
(20)	$20 \cdot 15 + 21$	321
(20, 20, 15)	$20 \cdot 15 + 20 \cdot 21 + 15 \cdot 21 + 16$	1,051
(20, 15, 10)	$20 \cdot 15 + 15 \cdot 21 + 10 \cdot 16 + 11$	786

These are the numbers in the column `parameters` (Keras's `count_params` less the frozen offset weight).

There are $J = 19$ separate networks per run, one per development period, with no parameters shared between them.

4.3 Feature coding

Each covariate is dummy coded with the label that has the most reported claims as reference (the paper’s Section 3.3): $1 + 5 + 4 + 3 + 1 = 14$ columns of zeros and ones. The reference cell (Legal Representation Y, Injury Severity 1, Age 30-50, Passenger, Business use N) is $\mathbf{x} = \mathbf{0}$. None of the five covariates is continuous, so the MinMaxScaler (3.8) of the paper is not used here.

5 The loss as Keras sees it

5.1 Response and volume

Keras has no built-in loss with the weights of (3). Listing 2 gets it by transforming the data (the paper’s (6.2)). For every row put

$$y = \frac{C_{i,j}(\mathbf{x})}{\sqrt{C_{i,j-1}(\mathbf{x})}}, \quad w = \sqrt{C_{i,j-1}(\mathbf{x})},$$

and let the network output be $f(\mathbf{x})w$: the layer `Offset` multiplies $f(\mathbf{x})$ by the second input w through a weight frozen at 1. Then the ordinary mean squared error is

$$\text{mse} = \frac{1}{n} \sum_r (y_r - f(\mathbf{x}_r) w_r)^2 = \frac{1}{n} \sum_r \frac{(d_r - f(\mathbf{x}_r) c_r)^2}{c_r} = \frac{1}{n} L_j(f), \quad (7)$$

because $(y - fw)^2 = (d/\sqrt{c} - f\sqrt{c})^2 = (d - fc)^2/c$. Minimising Keras’s mse is minimising (3). In `nncl_fit()` the saved factor is recovered as `f_learn = mu / w`.

5.2 The unit of the payments

On SynthETIC the fit script divides the payments by $u = 10^6$ before it forms y and w . With $c' = c/u$ and $d' = d/u$,

$$y' = \frac{d/u}{\sqrt{c'/u}} = \frac{y}{\sqrt{u}}, \quad w' = \frac{w}{\sqrt{u}}, \quad \frac{(d' - fc')^2}{c'} = \frac{1}{u} \frac{(d - fc)^2}{c}. \quad (8)$$

The loss is divided by u and its minimiser is unchanged: f is a ratio and has no unit. What does change is the size of the gradient, which is also divided by u . That matters for plain SGD and not for the adaptive optimisers (Section 9.3). The script multiplies the saved losses by u again, so that all tables are in the payments’ unit.

6 Gradients

All optimisers use the gradient of the mean loss (7) on the training rows. Let n be the number of those rows and write, for row r , $a_r = a(\mathbf{x}_r)$ and $f_r = e^{a_r}$.

6.1 Derivative in the output

Since $\partial f_r / \partial a_r = f_r$,

$$\delta_r := \frac{\partial \text{mse}}{\partial a_r} = \frac{1}{n} \cdot \frac{2(d_r - f_r c_r)(-c_r f_r)}{c_r} = -\frac{2}{n} (d_r - f_r c_r) f_r. \quad (9)$$

The term $d_r - f_r c_r$ is the residual of the cell in payments: observed $C_{i,j}(\mathbf{x})$ less predicted. The division by c_r in the loss cancels against the c_r from the chain rule, so large and small cells enter the gradient through their raw residuals.

6.2 Output layer

$$\frac{\partial \text{mse}}{\partial \beta_0} = \sum_r \delta_r, \quad \frac{\partial \text{mse}}{\partial \beta_k} = \sum_r \delta_r z_k^{(M)}(\mathbf{x}_r). \quad (10)$$

6.3 Hidden layers (back-propagation)

Because $\tanh'(t) = 1 - \tanh^2(t)$, the derivative in the pre-activation of neuron k of the last hidden layer is

$$\delta_{r,k}^{(M)} = \delta_r \beta_k \left(1 - z_k^{(M)}(\mathbf{x}_r)^2\right), \quad (11)$$

and for the layers below, $m = M - 1, \dots, 1$,

$$\delta_{r,k}^{(m)} = \left(\sum_{k'} \delta_{r,k'}^{(m+1)} W_{k',k}^{(m+1)} \right) \left(1 - z_k^{(m)}(\mathbf{x}_r)^2\right). \quad (12)$$

The gradients of the weights and intercepts are

$$\frac{\partial \text{mse}}{\partial W_{k,l}^{(m)}} = \sum_r \delta_{r,k}^{(m)} z_l^{(m-1)}(\mathbf{x}_r), \quad \frac{\partial \text{mse}}{\partial b_k^{(m)}} = \sum_r \delta_{r,k}^{(m)}. \quad (13)$$

Note the factor β_k in (11): every gradient of a hidden layer is a sum of terms that each contain one of the output weights. This is the key to Section 7.

7 The chain-ladder start

This is the training called `cl_start` (run S3 of the machine-data script; your own design, not in the paper). It changes one thing: how the output neuron is initialised.

7.1 Definition

Keras's default initialisation, used by Listing 2, draws all weights at random (Glorot uniform) and sets all intercepts to zero. In particular $\beta_0 = 0$, so every factor starts near $f = e^0 = 1$ plus random noise from $\beta^\top \mathbf{z}$, and the network has to learn the level of the factor as well as its dependence on the features.

The chain-ladder start keeps the random hidden layers and sets the output neuron to

$$\beta = \mathbf{0}, \quad \beta_0 = \log \hat{f}_{j-1}^{\mathcal{T}}, \quad \hat{f}_{j-1}^{\mathcal{T}} = \frac{\sum_{\mathcal{T}_j} C_{i,j}(\mathbf{x})}{\sum_{\mathcal{T}_j} C_{i,j-1}(\mathbf{x})}, \quad (14)$$

the homogeneous factor (4) computed on the training rows $\mathcal{T}_j$ only (the first 90% of the learning cells, Section 8). In the code:

```
get_layer(model, "CL_factor") %>%
  set_weights(list(matrix(0, q[length(q)], 1), array(log(f_start))))
```

7.2 Property 1: the start is the chain ladder

With $\beta = \mathbf{0}$ the hidden layers are disconnected from the output, whatever their weights:

$$f(\mathbf{x}) = \exp\left(\log \hat{f}_{j-1}^{\mathcal{T}} + \mathbf{0}^\top \mathbf{z}^{(M)}(\mathbf{x})\right) = \hat{f}_{j-1}^{\mathcal{T}} \quad \text{for every } \mathbf{x}. \quad (15)$$

So before any training the network *is* the homogeneous chain ladder, and its training loss is the smallest loss any feature-free factor can reach on $\mathcal{T}_j$. The homogeneous model is nested in the network: it is the point (14) of the parameter space. Training can then be read as

$$\log f(\mathbf{x}) = \underbrace{\log \hat{f}_{j-1}^{\mathcal{T}}}_{\text{chain ladder}} + \underbrace{\left(\beta_0 - \log \hat{f}_{j-1}^{\mathcal{T}}\right) + \beta^\top \mathbf{z}^{(M)}(\mathbf{x})}_{\text{correction learned from the features}},$$

a chain ladder plus a feature-dependent correction that starts at zero.

7.3 Property 2: the gradient at the start

At the start $f_r = \hat{f}_{j-1}^{\mathcal{T}} =: \hat{f}$ for all rows, so (9) becomes $\delta_r = -\frac{2\hat{f}}{n}(d_r - \hat{f}c_r)$.

Intercept. From (10) and the definition (14) of $\hat{f}$,

$$\frac{\partial \text{mse}}{\partial \beta_0} = -\frac{2\hat{f}}{n} \sum_{r \in \mathcal{T}_j} (d_r - \hat{f}c_r) = -\frac{2\hat{f}}{n} \left(\sum_{\mathcal{T}_j} d_r - \hat{f} \sum_{\mathcal{T}_j} c_r \right) = 0. \quad (16)$$

The level is already optimal: the residuals of the chain ladder sum to zero on the rows it was computed from. This is why $\hat{f}$ is taken on the training rows and not on all learning cells; with the factor (4) of all cells the gradient (16) would not vanish.

Hidden layers. Every term of (11) contains a $\beta_k = 0$, hence $\delta_{r,k}^{(M)} = 0$, and by (12)-(13)

$$\frac{\partial \text{mse}}{\partial W^{(m)}} = \mathbf{0}, \quad \frac{\partial \text{mse}}{\partial \mathbf{b}^{(m)}} = \mathbf{0} \quad \text{for all } m. \quad (17)$$

The hidden layers receive no signal in the first step.

Output weights. The only non-zero part of the gradient is

$$\frac{\partial \text{mse}}{\partial \beta_k} = -\frac{2\hat{f}}{n} \sum_{r \in \mathcal{T}_j} (d_r - \hat{f}c_r) z_k^{(M)}(\mathbf{x}_r). \quad (18)$$

Because the residuals sum to zero, (18) is $-2\hat{f}$ times the empirical covariance between the chain-ladder residual $d_r - \hat{f}c_r$ and the activation $z_k^{(M)}$ of the (still random) neuron k . The network moves away from the chain ladder exactly as far as some random feature of $\mathbf{x}$ is correlated with the chain-ladder residuals. If the residuals were unrelated to the covariates, all of (18) would be noise around zero and the start would already be a stationary point.

7.4 Property 3: curvature in the level

Differentiating (9) once more, $\partial\delta_r/\partial a_r = \frac{2}{n}f_r(2f_rc_r - d_r)$. Summed over the rows at the start,

$$\frac{\partial^2 \text{mse}}{\partial \beta_0^2} = \frac{2\hat{f}}{n} \left(2\hat{f} \sum c_r - \sum d_r \right) = \frac{2\hat{f}^2}{n} \sum_{\mathcal{T}_j} c_r > 0. \quad (19)$$

Together with (16): in the direction of the level the start is a strict minimum. In the directions of the hidden weights the loss is flat to first order (17), and it slopes only in the directions of the output weights (18). The start is therefore a point where the chain ladder is optimal as a level and where improvement can only come from the features.

7.5 What the first steps do

Step 1. Only β changes, by (16)-(18). With Adam the first step of each coordinate is $-\eta \text{sign}(g)$ (Section 9.2), so every β_k moves by $\pm\eta = \pm 0.001$ in the direction that reduces the correlation (18). With plain SGD it moves by $-\eta$ times (18).

Step 2 onwards. Now $\beta_k \neq 0$, the factor β_k in (11) no longer kills the signal, and the hidden weights start to move: the network begins to reshape its features z_k so that they pick up the pattern of the chain-ladder residuals. The intercept gradient $\sum_r \delta_r$ is no longer zero either, because f_r now differs across rows, so the level is re-balanced as the features come in.

Early in training the correction is small. After t Adam steps each parameter has moved by at most about $t\eta$, so $|\log f(\mathbf{x}) - \log \hat{f}|$ is at most of the order $t\eta(1 + \sum_k |z_k|)$. With $\eta = 0.001$ a network that is stopped after a few dozen steps is a chain ladder with corrections of a few percent. This is what the fitted runs show: the in-sample losses of the `cl_start` runs lie within about 0.4% of the homogeneous loss.

7.6 Comparison with the paper's random start

From the random start the level must be learned. For $j = 1$ the target is $\log \hat{f}_0 = \log 2.53 = 0.93$, starting from $\beta_0 = 0$. Each step moves each parameter by about $\eta = 0.001$ (RMSprop, Adam), so the intercept alone covers at most 0.1 in 100 steps; the rest has to come from the output weights, $\sum_k \beta_k z_k$, where each of the q terms can also contribute about $0.1 |z_k|$. Roughly,

$$\max |\log f| \text{ after 100 steps} \approx 100\eta \left(1 + \sum_{k=1}^q |z_k| \right) \leq 0.1(1 + q).$$

With $q = 20$ the level 0.93 is within reach, with $q = 5$ ($0.1 \cdot 6 = 0.6$) it is not. This is consistent with the paper run $q = 5$ being far worse (+479%) than $q = 10$ and $q = 20$. The bound is an order of magnitude, not an exact statement: RMSprop's first steps are about 3η and the z_k move as well.

In the paper this problem does not arise, because there one epoch is about 184 mini-batch steps and 100 epochs are about 18,400 steps. On SynthETIC the batch of 10,000 holds all rows (at most 2,656 training rows), so one epoch is one step and a paper run is 100 steps. In the late development periods the factor is close to 1 and the level is easy, but the random $\beta^\top \mathbf{z}$ at the start is noise of a few percent around it that 100 steps do not remove; multiplied over up to 19 periods in (2) it becomes the large over-reserving of the paper runs.

8 Early stopping and the stopping rule of the chain-ladder start

8.1 Training and validation rows

The learning cells are ordered by accident year and then by the features. Keras's `validation_split = 0.1` takes the **last** 10% of the rows as validation rows, without shuffling:

$$\mathcal{T}_j = \text{the first } \lfloor 0.9 n_j \rfloor \text{ rows of } \mathcal{L}_j, \quad \mathcal{V}_j = \mathcal{L}_j \setminus \mathcal{T}_j.$$

So the validation rows are the cells of the latest accident years of each development period (for $j = 1$ accident years 18-19, for $j = 2$ accident years 17-18, for $j = 10$ accident year 10). It is a hold-out in time, not a random one. The gradient uses $\mathcal{T}_j$ only; with a batch size above $|\mathcal{T}_j|$ every epoch is one full-batch gradient step.

8.2 Early stopping

The `cl_start` training (and S2) runs up to 500 epochs and monitors the validation loss $L^\mathcal{V}(t) = \frac{1}{|\mathcal{V}_j|} \sum_{\mathcal{V}_j} (y - f_t w)^2$ after every epoch t . With patience 20 it stops at the first epoch t for which $L^\mathcal{V}$ has not improved for 20 epochs, and restores the weights of

$$t^* = \arg \min_{1 \leq t \leq t_{\text{stop}}} L^\mathcal{V}(t).$$

This is the pair `epochs_used / epochs_run =  $t^*/t_{\text{stop}}$` in the tables.

8.3 Keeping the start

Keras's callback only compares the epochs $t \geq 1$ with each other; it never considers the start $t = 0$. `nnc1_fit()` adds that comparison:

```
if (param$early_stop && !is.null(f_start) &&
    min(val_loss, na.rm = TRUE) >= mean((y[valid] - mu_start)^2)) {
  set_weights(model, start)
  epochs_used <- 0
}
```

that is,

$$\theta_{\text{final}} = \begin{cases} \theta_0 \text{ (the chain ladder)} & \text{if } \min_{t \geq 1} L^\mathcal{V}(t) \geq L^\mathcal{V}(0), \\ \theta_{t^*} & \text{otherwise.} \end{cases} \quad (20)$$

Consequence. By construction $L^\mathcal{V}(\theta_{\text{final}}) \leq L^\mathcal{V}(\theta_0)$: on the validation rows the fitted network is never worse than the homogeneous factor $\hat{f}_{j-1}^\mathcal{T}$. A development period where the features do not help on the latest accident years falls back to the chain ladder exactly (`epochs_used = 0`). In your S3 run (one layer of 20, Adam) this happens for $j = 1, 11, 15, 16, 17, 18, 19$; the epochs used for $j = 1, \dots, 19$ are 0, 169, 62, 16, 62, 23, 21, 15, 1, 8, 0, 4, 1, 18, 0, 0, 0, 0, 0.

The guarantee is about the validation rows of the same triangle. It is not a guarantee about the reserve: the validation loss is used to choose t^* , so it is optimistic, and the reserve depends on the product (2) over development periods.

9 Optimisers

All ten optimisers minimise the same training loss; they differ in how the gradient $g_t = \nabla_{\theta} \text{mse}(\theta_{t-1})$ is turned into a step. Because every epoch is one full batch, there is no sampling noise in g_t here: “SGD” is plain gradient descent. Operations on vectors below are component by component; $\epsilon = 10^{-7}$ is Keras’s small constant that keeps denominators positive (implementations differ in whether it is added inside or outside the square root).

9.1 Update rules

optimiser	state	update	η
SGD	none	$\theta_t = \theta_{t-1} - \eta g_t$	0.001
SGD with momentum	$v_t = \mu v_{t-1} - \eta g_t$	$\theta_t = \theta_{t-1} + v_t, \mu = 0.9$	0.001
Nesterov	$v_t = \mu v_{t-1} - \eta g_t$	$\theta_t = \theta_{t-1} + \mu v_t - \eta g_t$	0.001
Adagrad	$A_t = A_{t-1} + g_t^2, A_0 = 0.1$	$\theta_t = \theta_{t-1} - \eta g_t / \sqrt{A_t + \epsilon}$	0.01
Adadelata	$E_t = \rho E_{t-1} + (1 - \rho) g_t^2,$ $D_t = \rho D_{t-1} + (1 - \rho) \Delta_t^2$	$\Delta_t = -\sqrt{\frac{D_{t-1} + \epsilon}{E_t + \epsilon}} g_t, \theta_t = \theta_{t-1} + \eta \Delta_t,$ $\rho = 0.95$	1
RMSprop	$v_t = \rho v_{t-1} + (1 - \rho) g_t^2$	$\theta_t = \theta_{t-1} - \eta g_t / \sqrt{v_t + \epsilon}, \rho = 0.9$	0.001
Adam	$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$ $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$	$\theta_t = \theta_{t-1} - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$	0.001
AdamW	as Adam	Adam’s step, and θ also shrunk by $\eta \lambda \theta_{t-1}, \lambda = 0.004$	0.001
Adamax	m_t as Adam, $u_t = \max(\beta_2 u_{t-1}, g_t)$	$\theta_t = \theta_{t-1} - \frac{\eta}{1 - \beta_1^t} m_t / (u_t + \epsilon)$	0.001
Nadam	as Adam	Adam with a Nesterov look-ahead in m_t	0.001

For Adam, $\hat{m}_t = m_t / (1 - \beta_1^t)$ and $\hat{v}_t = v_t / (1 - \beta_2^t)$ are the bias-corrected moments, with $\beta_1 = 0.9$ and $\beta_2 = 0.999$ (these β ’s are the optimiser’s constants, not the output weights of the network).

Two remarks on the results. AdamW and Adam give almost identical fits because the extra shrinkage per step is $\eta \lambda \theta = 4 \cdot 10^{-6} \theta$, negligible over a few hundred steps. And the learning rates are not comparable across the families: SGD’s η multiplies the raw gradient, whereas in the adaptive methods η is roughly the step length itself.

9.2 Step size of the adaptive methods

At $t = 1$ Adam has $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$, so the step is

$$\theta_1 - \theta_0 = -\eta \frac{g_1}{|g_1| + \epsilon} \approx -\eta \text{sign}(g_1),$$

and RMSprop has $v_1 = (1 - \rho) g_1^2$, so its first step is $-\eta g_1 / \sqrt{0.1 g_1^2} \approx -3.16 \eta \text{sign}(g_1)$. Later steps stay of the order η as long as the gradient keeps its sign, and shrink when it starts to alternate. A parameter can therefore move by about $t\eta$ in t steps and not more, which is the bound used in Section 7.6.

9.3 Why the unit of the payments matters for SGD only

By (8), changing the unit multiplies the loss and hence g_t by a constant $\kappa = 1/u$. In RMSprop, Adam, Adamax, Nadam and Adadelata the step is a ratio of the form $g / \sqrt{\text{average of } g^2}$, in which κ cancels (apart from ϵ): these methods take the same steps in francs and in millions. Adagrad cancels

κ up to its starting value A_0 . In SGD and its momentum variants the step is $\eta \kappa g$: in the original unit the gradients are of the order 10^6 times larger and the first step throws the exponential output to infinity. Fitting in millions puts SGD on a scale where a learning rate of 10^{-3} is stable.

That the unit should not matter for RMSprop is not quite what happened in practice: the refit in millions (which also ran with a different thread setting) moved the reserve of the paper run $q = 20$ from +34.1% to +43.4% bias, with the total loss unchanged to five digits. The reason is that after exactly 100 steps the factors of single cells still differ by a few percent between two numerically slightly different trajectories, and the reserve multiplies up to 19 of them. It is a sign that the 100-step fit has not converged, not that the unit carries information.

10 The balance correction (S4)

After training, the fitted factors need not reproduce the total of the payments: in general $\sum_{\mathcal{T}_j} f(\mathbf{x}) C_{i,j-1}(\mathbf{x}) \neq \sum_{\mathcal{T}_j} C_{i,j}(\mathbf{x})$. The chain ladder has this **balance property** by (16). A network loses it for two reasons. Early stopping ends the descent before a minimum is reached. And even at a minimum, the condition that the intercept gradient (10) vanishes is

$$\sum_r \delta_r = 0 \iff \sum_r (d_r - f_r c_r) f_r = 0,$$

residuals weighted by the fitted factors, which is the balance $\sum_r (d_r - f_r c_r) = 0$ only when f is constant. (A squared loss with a log link is not the canonical pair; under a Poisson deviance with the log link the intercept condition would be the balance property itself.)

S4 (your own design) restores the balance of S3 by rescaling the factors of each development period by

$$c_j = \frac{\sum_{\mathcal{T}_j} C_{i,j}(\mathbf{x})}{\sum_{\mathcal{T}_j} f(\mathbf{x}) C_{i,j-1}(\mathbf{x})}, \quad f^{\text{bal}}(\mathbf{x}) = c_j f(\mathbf{x}), \quad (21)$$

so that $\sum_{\mathcal{T}_j} f^{\text{bal}}(\mathbf{x}) C_{i,j-1}(\mathbf{x}) = \sum_{\mathcal{T}_j} C_{i,j}(\mathbf{x})$. Equivalently, the volume-weighted average factor

$$\frac{\sum_{\mathcal{T}_j} f^{\text{bal}}(\mathbf{x}) C_{i,j-1}(\mathbf{x})}{\sum_{\mathcal{T}_j} C_{i,j-1}(\mathbf{x})} = \hat{f}_{j-1}^{\mathcal{T}}$$

equals the homogeneous factor (the average (3.9) of the paper, Section 13). In terms of the network, (21) adds $\log c_j$ to β_0 and leaves the dependence on the features untouched. The losses of S4 are recomputed with f^{bal} ; they can be slightly higher than those of S3, because (21) is a balance condition and not a least squares step. At the chain-ladder start itself $c_j = 1$.

11 The nagging predictor

11.1 Definition

Two networks that differ only in the seed (the random initial hidden weights) end at different parameters, because the loss is not convex in θ and early stopping ends the descent on the way. The **nagging predictor** (network aggregating; Richman and Wuthrich 2020) averages the predictions of K such networks. Here it is formed separately for every development period j :

$$\bar{f}_{j-1}(\mathbf{x}) = \frac{1}{K} \sum_{k=1}^K f_{j-1}^{(k)}(\mathbf{x}), \quad K = 10, \quad (22)$$

where $f_{j-1}^{(k)}$ is the network of development period j fitted with seed k . The average is taken of the factors themselves and not of their logarithms, because the factor is the model's mean, $\mathbb{E}[C_{i,j} | C_{i,j-1}] = f C_{i,j-1}$, and an average of means is again a mean prediction. The reserves then use $\bar{f}$ in place of f in (2) and (26).

In the analysis script the networks of a block of 10 seeds are averaged cell by cell (**f_learn** on the learning cells, **f_diag** on the diagonal cells); with 20 seeds there are two disjoint nagging predictors per combination (seeds 2026-2035 and 2036-2045).

11.2 Why averaging helps

Given the data $\mathcal{D}$, the K fits are independent and identically distributed draws, because only the seed differs. For a fixed cell,

$$\mathbb{E}[\bar{f} | \mathcal{D}] = \mathbb{E}[f^{(1)} | \mathcal{D}], \quad \text{Var}(\bar{f} | \mathcal{D}) = \frac{1}{K} \text{Var}(f^{(1)} | \mathcal{D}). \quad (23)$$

Averaging removes the part of the error that comes from the seed, at the rate $1/K$, and leaves untouched what all seeds have in common: the bias of the method and the randomness of the data. A nagging predictor is therefore expected to sit at the centre of the cloud of single-seed results, not outside it.

There is also an exact statement about the loss. The loss (3) is convex in f for every cell, so by Jensen's inequality

$$L_j(\bar{f}) \leq \frac{1}{K} \sum_{k=1}^K L_j(f^{(k)}), \quad (24)$$

on the learning cells, on the validation rows, and on any other set of cells: the nagging predictor is never worse than its members are on average. The gain is the spread of the members,

$$\frac{1}{K} \sum_k L_j(f^{(k)}) - L_j(\bar{f}) = \sum_r c_r \cdot \frac{1}{K} \sum_k (f^{(k)}(\mathbf{x}_r) - \bar{f}(\mathbf{x}_r))^2,$$

the volume-weighted variance of the factors across the seeds. Where the seeds agree (the chain-ladder start, stopped early), the gain is small; where they disagree (the paper's training), it is large.

11.3 The reserve of a nagging predictor

The reserve multiplies the averaged factors over the development periods, $\prod_j \bar{f}_j(\mathbf{x})$. This is not the average of the single networks' products: if the seeds were paired across periods,

$$\frac{1}{K} \sum_k \prod_j f_j^{(k)}(\mathbf{x}) \neq \prod_j \frac{1}{K} \sum_k f_j^{(k)}(\mathbf{x})$$

in general, the difference being the covariances of the factors of different periods across seeds. Averaging per development period, as (22) does, is the natural choice here because the 19 networks are separate models with nothing in common but the seed number.

With the chain-ladder start, every member starts at the same level $\hat{f}_{j-1}^{\mathcal{T}}$ and, where rule (20) keeps the start, all K members are identical; the nagging predictor is then the chain-ladder factor itself.

12 Reserves and the zero claims features

12.1 Part 1: cells with a positive diagonal

For accident years $i \geq 2$ and cells with $C_{i,I-i}(\mathbf{x}) > 0$, (2) gives the reserve

$$\hat{R}_1 = \sum_{i=2}^I \sum_{\mathbf{x}: C_{i,I-i}(\mathbf{x}) > 0} C_{i,I-i}(\mathbf{x}) \left(\prod_{j=I-i}^{J-1} f_j(\mathbf{x}) - 1 \right). \quad (25)$$

Network j (for $j = 1, \dots, J$) delivers f_{j-1} , so a cell whose latest development year is $m = I - i$ uses the networks $m + 1, \dots, J$. In the analysis script this is the matrix `f_diag` (one column per network, evaluated at the feature of each diagonal cell) and

```
exp(rowSums(ifelse(outer(m_diag, 1:(n_ay - 1), "<"), log(f_diag), 0)))
```

which is $\prod_{j>m} f_{j-1}(\mathbf{x})$ computed on the log scale.

12.2 Part 2: zero claims features

A cell with $C_{i,I-i}(\mathbf{x}) = 0$ cannot be developed by a factor: $0 \cdot f = 0$. Yet such cells will have payments (claims reported late, or reported and not yet paid). The paper's Section 4.2 estimates them from how the zero cells of earlier accident years developed.

For accident year i , with latest development year $m = I - i$, the **zero set** of an earlier accident year $k < i$ is

$$\mathcal{X}_k^{(i)} = \{\mathbf{x} : C_{k,I-i}(\mathbf{x}) \leq 0\},$$

the features that had paid nothing by the same development year. The first factor relates what those features paid one year later to the total volume of the earlier accident years:

$$g_{I-i}^{(i)} = \frac{\sum_{k < i} \sum_{\mathbf{x} \in \mathcal{X}_k^{(i)}} C_{k,I-i+1}(\mathbf{x})}{\sum_{k < i} C_{k,I-i}},$$

and the later factors, for $I - i < j < J$, are chain-ladder factors of the zero set itself, over the accident years for which development year $j + 1$ is observed:

$$g_j^{(i)} = \frac{\sum_{k \leq I-j-1} \sum_{\mathbf{x} \in \mathcal{X}_k^{(i)}} C_{k,j+1}(\mathbf{x})}{\sum_{k \leq I-j-1} \sum_{\mathbf{x} \in \mathcal{X}_k^{(i)}} C_{k,j}(\mathbf{x})}.$$

The predicted ultimate of the zero claims features of accident year i is the total diagonal volume times these factors, $C_{i,I-i} \prod_{j=I-i}^{J-1} g_j^{(i)}$, and the reserve (the paper's (5.1)) is

$$\hat{R} = \hat{R}_1 + \underbrace{\sum_{i=2}^I \left(C_{i,I-i} \prod_{j=I-i}^{J-1} g_j^{(i)} - \sum_{\mathbf{x}: C_{i,I-i}(\mathbf{x}) \leq 0} C_{i,I-i}(\mathbf{x}) \right)}_{\hat{R}_2}. \quad (26)$$

The last sum is zero on SynthETIC (no recoveries, so no negative cumulative payments). Note that $\hat{R}_2$ does not depend on the networks: it is 75.3 million for every run, against 44.8 million truly outstanding on the 16 zero cells.

12.3 The three rules of the code

`nncl_zero_claims_factors()` applies three rules that the paper does not spell out; they are the same on SynthETIC as on the machine data.

1. The zero set is defined by $C \leq 0$ and not $C = 0$, because recoveries can make a cumulative payment negative. On SynthETIC this is the same as $C = 0$.
2. 0/0 gives the factor 1 (a zero set that did not develop). A first factor $g_{I-i}^{(i)} = 0$ gives the ultimate 0, whatever the later factors are.
3. A positive numerator over a zero denominator would give an infinite factor. On the machine data it takes the ratio pooled over the lines of business. SynthETIC is one portfolio, so there is nothing to pool over and such a factor stays infinite. It occurs for $g_7^{(i)}$ of accident years $i = 14, 15, 16$ and is harmless there by rule 2, because their first factor is 0. Otherwise `nncl_reserves()` stops (`stopifnot(all(is.finite(res$reserve)))`).

13 Reading the fitted factors: marginal averages

The figures of the CL factor sensitivities (the paper’s Figures 8-9) show, for each label ℓ of a covariate, the volume-weighted average factor over the learning cells with that label (the paper’s (3.9)):

$$\bar{f}_{j-1}(\ell) = \frac{\sum_{(i,\mathbf{x}) \in \mathcal{L}_j, \mathbf{x} \ni \ell} f_{j-1}(\mathbf{x}) C_{i,j-1}(\mathbf{x})}{\sum_{(i,\mathbf{x}) \in \mathcal{L}_j, \mathbf{x} \ni \ell} C_{i,j-1}(\mathbf{x})},$$

and the dotted line is the same average over all learning cells, the factor that the network implies for the portfolio as a whole. With the balance correction (21) that portfolio average equals the homogeneous factor on the training rows.

14 Where each formula is in the code

Formula	Code
cells $C_{i,j}(\mathbf{x})$, learning cells $\mathcal{L}_j$	fit script, sections “cells” and “feature pre-processing” (<code>cum</code> , <code>learn_rows</code>)
homogeneous factor (4) and its loss	fit script, section “homogeneous model” (<code>homogeneous\$f_hom</code> , <code>\$loss</code>)
network (5)-(6), parameter count	<code>nncl_model()</code> in <code>R/nn_chain_ladder.R</code> (<code>q</code> a vector)
response and volume (7), unit (8)	fit script, loop “CL factor networks” (<code>y</code> , <code>w</code> , <code>units</code>)
chain-ladder start (14)	fit script (<code>f_start</code>), <code>nncl_model()</code> (<code>set_weights</code>)
early stopping, rule (20)	<code>nncl_fit()</code> (<code>callback_early_stopping</code> , <code>epochs_used</code>)
optimisers	<code>nncl_optimizer()</code> ; learning rates in <code>config.yml</code> , <code>nncl:synthetic: optimizers</code>
balance correction (21)	fit script, block “S4”
nagging predictor (22)	analysis script, block “nagging predictor” (<code>nag</code> , pseudo-fits in <code>fits</code>)
zero claims factors $g_j^{(i)}$	<code>nncl_zero_claims_factors()</code>
reserves (25)-(26)	analysis script (<code>f_prod</code>), <code>nncl_reserves()</code>
marginal averages	analysis script, block “Figs. 8-9”

15 Numerical check of the formulas

All checks are on the SynthETIC cells in millions, development period $j = 2$ ($n_2 = 2,952$ learning cells, 2,656 training rows) unless stated otherwise, in base R without Keras.

Check	Result
(4): $\sum d / \sum c$ against the saved <code>f_hom</code> and against a numerical minimiser of (3)	1.326284 in all three
(7): $n \cdot \text{mse}$ at $\hat{f}$ against the saved homogeneous loss	9,755,989,751 in both
(8): loss in the original unit divided by u against the loss in millions	3.304875 in both
(9)-(13): gradient against central finite differences, random weights, hidden layers (20)	maximum relative error $1.1 \cdot 10^{-7}$
(9)-(13): the same, hidden layers (20, 20, 15)	maximum relative error $6.5 \cdot 10^{-8}$
(15): factor at the chain-ladder start, all training rows	1.326688 for every row ($= \hat{f}_1^{\mathcal{T}}$)
(16): $\partial \text{mse} / \partial \beta_0$ at the start	$-9 \cdot 10^{-15}$
(17): largest gradient of a hidden weight or intercept at the start	exactly 0
(18): $\partial \text{mse} / \partial \beta_k$ against the residual formula	maximum difference $2 \cdot 10^{-15}$ (largest gradient 0.686)
(19): second derivative in β_0 against a finite difference	48.21369 in both
parameter counts (20), (20, 20, 15), (20, 15, 10) against Keras	321, 1,051, 786 in both
rule (20): the S3 networks with <code>epochs_used = 0</code> against $\hat{f}_{j-1}^{\mathcal{T}}$	maximum relative deviation $1.8 \cdot 10^{-7}$ (single precision of Keras)

The optimiser formulas of Section 9.1 and the nagging statements of Section 11 are standard results and were not recomputed; the update rules are stated as the methods are published and Keras's implementation may differ in details such as the place of ϵ .